

PROJECT-BASED LEARNING

PHASE-I EVALUATION

MemDB: In-Memory Key-Value Cache

Team ID: DSCPP-III-2026-T229

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T229

Semester: 3rd

Domain: Systems / Software

Project: MemDB

Team Members

1. Anish Mandal (ML1) – 2028642
2. Arnav Rawat (ML4) – 2028398
3. Priyanshi (DS2) – 2028468
4. Suhani Rawat (DS1) – 2028504

Mentor

Dr. Ravi Sharma

Number of Interactions: 1

Arnav Rawat is the team lead.

Problem Statement & Motivation

Problem Statement

Applications often access the same data repeatedly. Fetching data from slower storage can increase access time. MemDB provides a lightweight **in-memory key-value cache** that keeps frequently accessed data in memory while managing lookup, capacity, expiration and eviction.

Why MemDB?

- ▶ Reduce repeated data access.
- ▶ Understand caching internally.
- ▶ Apply data structures to a real problem.
- ▶ Study cache eviction strategies.
- ▶ Measure performance using benchmarks.

Project Focus

- ▶ Hash-based key lookup.
- ▶ Linked-list collision handling.
- ▶ LRU and FIFO eviction.
- ▶ TTL-based expiration.
- ▶ Memory and cache statistics.

Goal: Build and evaluate a lightweight in-memory cache using fundamental data structures and cache policies.

Objectives

Project Objectives

1. Develop a functional lightweight in-memory key-value cache.
2. Implement hash-based key lookup with collision handling.
3. Implement LRU and FIFO cache eviction policies.
4. Implement TTL-based expiration using a min-heap.
5. Apply C data structures and C++ object-oriented abstractions.
6. Track memory usage and cache statistics.
7. Evaluate the implementation using controlled workloads and measurable metrics.

Key Objective

Build and experimentally evaluate a lightweight in-memory cache using explicitly implemented data structures and cache policies.

Proposed Solution

Approach

1. Store key-value entries in memory.
2. Use a hash table for average $O(1)$ key lookup.
3. Handle collisions using linked lists.
4. Maintain order for eviction policies.
5. Use a min-heap for earliest TTL expiration.
6. Track memory consumption and statistics.
7. Provide a C++ interface over the structures.

The listed features form the proposed implementation scope.

Core Features

- ▶ SET / GET
- ▶ DELETE / CLEAR
- ▶ LRU eviction
- ▶ FIFO eviction
- ▶ TTL expiration
- ▶ Configurable capacity
- ▶ Hit / miss statistics
- ▶ Memory usage tracking
- ▶ Benchmarking

Integration of PBL Course Concepts

Data Structures in C

Hash tables, linked lists, doubly linked lists, min-heaps and dynamic memory management support cache storage, collision handling, LRU ordering and TTL management.

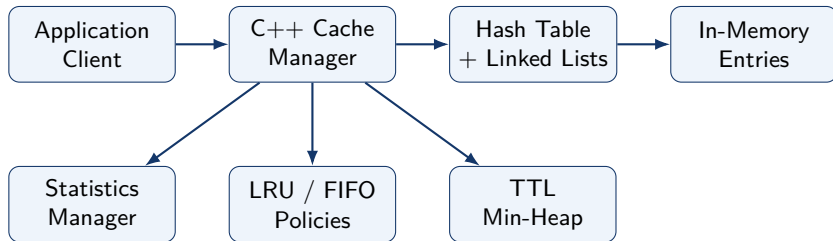
OOPs with C++

Classes, encapsulation, abstraction and polymorphism organize cache management and eviction-policy implementations.

Integration

C provides low-level data structures, while C++ provides higher-level cache management and policy abstraction.

System Architecture / Workflow



Core Components

- ▶ Hash table
- ▶ Collision lists
- ▶ LRU / FIFO policies
- ▶ TTL min-heap
- ▶ Statistics

Workflow

- ▶ Application sends GET / SET.
- ▶ Hash table locates the entry.
- ▶ Policy state is updated.
- ▶ TTL manager tracks expiration.
- ▶ Statistics are recorded.

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C and C++
- ▶ Core data structures: C
- ▶ Cache management / OOP: C++
- ▶ Development environment: Windows
- ▶ Version control: Git / GitHub
- ▶ Testing: Custom test programs and workloads

Development Methodology

1. Requirement analysis
2. Data structure design
3. Cache architecture
4. Core implementation
5. Policy and TTL implementation
6. Testing
7. Benchmarking
8. Evaluation

The project focuses on the in-memory cache and its underlying algorithms and data structures.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Problem and scope identified.
- ▶ Core cache operations defined.
- ▶ Hash table selected for lookup.
- ▶ Linked lists selected for collisions.
- ▶ Doubly linked list selected for LRU.
- ▶ Min-heap selected for TTL.
- ▶ LRU and FIFO selected as policies.
- ▶ Benchmarking metrics identified.

Team Contribution

Member	Role	Contribution
Anish Mandal	Developer	Core implementation, data structures and system design
Arnav Rawat	Team Lead	Coordination, architecture and implementation
Priyanshi	Developer	Testing, cache operations and evaluation
Suhani Rawat	Developer / Documentation	Documentation, testing and project support

Roadmap, Outcomes & References

Roadmap

1. Phase-I: Requirements & architecture
2. Phase-II: Core data structures
3. Phase-III: LRU, FIFO & TTL
4. Phase-IV: Testing & benchmarking
5. Phase-V: Visualization & evaluation

Expected Outcomes

- ▶ Working in-memory cache
- ▶ DSA implementation in C
- ▶ OOP-based management in C++
- ▶ LRU, FIFO and TTL
- ▶ Statistics and memory tracking
- ▶ Benchmark results

References

1. Cormen et al., *Introduction to Algorithms*
2. Kernighan & Ritchie, *The C Programming Language*
3. Bjarne Stroustrup, *The C++ Programming Language*

Thank You

Questions & Suggestions